Lab 18 — Container Security with Docker

> **SY0-701 Objective 3.1** — Compare and contrast security implications of different architecture models
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Show why a default container is over-privileged, then run the same workload as a non-root user on a read-only filesystem with no Linux capabilities, resource limits and a vulnerability scan.

You'll build: A custom non-root image, a hardened container (`--read-only`, `--cap-drop ALL`, `no-new-privileges`), a resource-capped container that survives a fork bomb, and a Trivy scan report.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; Docker is installed in step 1 if the playground does not already have it. Images are pulled from Docker Hub.

Background

- **Containerisation** isolates processes with kernel **namespaces** (what a process can see) and **cgroups** (what it can consume), but every container **shares the host kernel** — isolation is weaker than a virtual machine with its own kernel.
- By default a container's main process runs as **root** with a set of Linux **capabilities** such as `CAP_CHOWN`, `CAP_NET_RAW` and `CAP_SETUID`. If the application is compromised, the attacker inherits all of them.
- Hardening follows least privilege: a non-root `USER` in the image, `--cap-drop ALL`, `--security-opt no-new-privileges`, a read-only root filesystem and never `--privileged`.
- **Resource limits** (`--memory`, `--cpus`, `--pids-limit`) stop one container exhausting the host — a denial-of-service control and an availability consideration.
- **Image scanning** finds known CVEs in the packages baked into an image; old base images carry many unpatched vulnerabilities.

Steps

1. Install Docker and check the engine

Install Docker only if it is missing, plus `capsh` for decoding capability sets and `jq`.

```
apt-get update -qq
command -v docker >/dev/null 2>&1 || DEBIAN_FRONTEND=noninteractive apt-get install -y docker.io
DEBIAN_FRONTEND=noninteractive apt-get install -y libcap2-bin jq curl
systemctl start docker 2>/dev/null || true
docker version --format 'Client {{.Client.Version}} / Server {{.Server.Version}}'
mkdir -p /root/lab18 && cd /root/lab18
```

2. Run a default container and inspect its privileges

Start a stock nginx container and look at who it runs as and what capabilities it holds.

```
docker run -d --name web-default nginx:alpine
sleep 2
docker exec web-default id
docker exec web-default grep CapEff /proc/1/status
capsh --decode=$(docker exec web-default awk '/CapEff/{print $2}' /proc/1/status)
```

```
uid=0(root) gid=0(root) groups=0(root),...
CapEff: 00000000a80425fb
0x00000000a80425fb=cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,ca
p_setuid,...
```

3. Compare container isolation with the host

The container has its own PID and network namespaces, but the same kernel as the host.

```
echo "Host kernel: $(uname -r)   Container kernel: $(docker exec web-default uname -r)"
docker exec web-default ps
PID=$(docker inspect -f '{{.State.Pid}}' web-default)
echo "Container PID 1 is host PID $PID"
readlink /proc/1/ns/pid /proc/$PID/ns/pid
readlink /proc/1/ns/net /proc/$PID/ns/net
```

The namespace IDs differ (isolation), the kernel version is identical (shared kernel — a kernel exploit escapes every container).

4. Build an image that runs as a non-root user

Write a Dockerfile with a dedicated unprivileged account and a `USER` instruction.

```
cd /root/lab18
cat > Dockerfile <<'EOF'
FROM alpine:3.20
RUN addgroup -S -g 10001 app && adduser -S -D -H -u 10001 -G app app
USER 10001:10001
CMD ["sleep", "3600"]
EOF
docker build -q -t lab18/app:1.0 .
docker run --rm lab18/app:1.0 id
```

```
uid=10001(app) gid=10001(app) groups=10001(app)
```

5. Harden the container: read-only, no capabilities, no new privileges

Even root inside this container is powerless: the filesystem is read-only and every capability is dropped.

```
docker run -d --name app-hardened --read-only --tmpfs /tmp:rw,noexec,nosuid,size=16m \
  --cap-drop ALL --security-opt no-new-privileges:true lab18/app:1.0
sleep 1
docker exec app-hardened id -u
docker exec -u 0 app-hardened touch /etc/pwned 2>&1 || echo "BLOCKED: root filesystem is
```

```
read-only"
docker exec app-hardened sh -c 'touch /tmp/scratch && echo "tmpfs /tmp is writable"'
docker exec -u 0 app-hardened chown 0 /tmp/scratch 2>&1 || echo "BLOCKED: root has no
CAP_CHOWN"
docker exec app-hardened awk '/CapEff/{print "CapEff:", $2}' /proc/1/status
```

```
10001
touch: /etc/pwned: Read-only file system
BLOCKED: root filesystem is read-only
tmpfs /tmp is writable
chown: /tmp/scratch: Operation not permitted
BLOCKED: root has no CAP_CHOWN
CapEff: 0000000000000000
```

6. Apply resource limits and survive a fork bomb

Cap memory, CPU and the number of processes, then try to spawn 60 processes inside a 30-process limit.

```
docker run -d --name app-capped --memory 64m --cpus 0.5 --pids-limit 30 lab18/app:1.0
sleep 1
docker inspect -f 'Memory={{.HostConfig.Memory}} NanoCpus={{.HostConfig.NanoCpus}}
PidsLimit={{.HostConfig.PidsLimit}}' app-capped
docker exec app-capped sh -c 'for i in $(seq 1 60); do sleep 30 & done' 2>&1 | tail -2
echo "Processes in container: $(docker exec app-capped ps -o pid= 2>/dev/null | wc -l)"
docker stats --no-stream app-capped
```

The loop fails with `can't fork: Resource temporarily unavailable` once the limit is reached, while the host stays responsive.

7. Audit default versus hardened settings

Compare the runtime security settings of all three containers side by side.

```
for c in web-default app-hardened app-capped; do
  docker inspect -f '{{.Name}}: user={{.Config.User}} readonly=
{{.HostConfig.ReadonlyRootfs}} capdrop={{.HostConfig.CapDrop}} secopt=
{{.HostConfig.SecurityOpt}} privileged={{.HostConfig.Privileged}} mem=
{{.HostConfig.Memory}} pids={{.HostConfig.PidsLimit}}' "$c"
done
```

```
/web-default: user= readonly=false capdrop=[] secopt=[] privileged=false mem=0 pids=<nil>
/app-hardened: user=10001:10001 readonly=true capdrop=[ALL] secopt=[no-new-
privileges:true] privileged=false mem=0 pids=<nil>
/app-capped: user=10001:10001 readonly=false capdrop=[] secopt=[] privileged=false
mem=67108864 pids=30
```

8. Scan images for known vulnerabilities

Install a pinned Trivy release from GitHub and compare an old base image with a current one.

```
cd /root/lab18
curl -sSL -o trivy.deb
https://github.com/aquasecurity/trivy/releases/download/v0.74.0/trivy_0.74.0_Linux-
```

```
64bit.deb
dpkg -i trivy.deb >/dev/null && trivy --version | head -1
trivy image --quiet --scanners vuln --severity HIGH,CRITICAL nginx:1.21-alpine | grep -E
'Total|^nginx' | head -5
trivy image --quiet --scanners vuln --severity HIGH,CRITICAL lab18/app:1.0 | grep -E
'Total' | head -3
```

The old `nginx:1.21-alpine` image reports many HIGH and CRITICAL CVEs; the image built on current Alpine reports few or none. Rebuilding on a patched base image is the fix.

Test it

```
docker exec app-hardened id -u
docker exec -u 0 app-hardened touch /etc/pwned 2>&1 | grep -c "Read-only file system"
docker inspect -f '{{.HostConfig.CapDrop}} {{.HostConfig.ReadonlyRootfs}}
{{.HostConfig.PidsLimit}}' app-hardened app-capped
```

Expected: `10001`, then `1` (write blocked even for root), then `[ALL] true <nil>` for the hardened container and `[] false 30` for the capped one.

Troubleshooting

Symptom	Fix
`toomanyrequests: You have reached your pull rate limit`	Docker Hub throttles anonymous pulls; wait a few minutes or restart the playground for a new IP.
`Conflict. The container name "/web-default" is already in use`	You are re-running the lab; stop and remove the old container with `docker rm -f web-default` (and likewise for the others).
`capsh: command not found`	Run `DEBIAN_FRONTEND=noninteractive apt-get install -y libcap2-bin`, or decode the hex mask online at a capability decoder.
Trivy hangs on `Downloading vulnerability DB`	The first scan downloads the DB from ghcr.io; allow a minute, or retry with `--timeout 10m`.

Challenge

Run `docker run --rm --privileged alpine:3.20 sh -c 'ls /dev | wc -l; grep CapEff /proc/1/status'` and compare the device count and capability mask with `app-hardened`. Then write a one-page container baseline that bans `--privileged`, requires a non-root `USER`, and states the default capability set your team allows.

Reflection

A developer argues that containers are "as isolated as VMs". Using what you saw in step 3 and step 5, explain the security implication of a shared kernel and which hardening flags reduce it.

Exam tips

- Containers share the host kernel; VMs have their own — a kernel exploit or a privileged container can escape to the host.

- Least privilege for containers: non-root user, drop capabilities, read-only filesystem, no `--privileged`.
- Resource limits (cgroups) are an availability control against denial of service from a runaway or compromised container.
- Image scanning finds known CVEs; it cannot find zero-days or insecure runtime settings.

References

- Docker Engine security — https://docs.docker.com/engine/security/
- OWASP Docker Security Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- CIS Docker Benchmark — https://www.cisecurity.org/benchmark/docker
- NIST SP 800-190 Application Container Security Guide — https://csrc.nist.gov/pubs/sp/800/190/final
- Trivy documentation — https://trivy.dev/
